Optional Supplement: Duplicates and `lower_bound`

Binary search when a value appears more than once

CSCI 235 – Data Structures

Hunter College, Fall 2026

Class 19 supplement (optional reading)

Why This Is Optional

Class 19 teaches the core idea: binary search on sorted data, $O(\log n)$. This supplement answers a question students often ask next:

"What if the same value appears several times?"

- Nothing later in CSCI 235 depends on this. Read it when you want to go further.
- It reuses the same `lo`, `hi`, `mid` ideas, with one change: the range is **half-open**.
- Real programs use it all the time: "how many orders were placed on this date?"

The Problem: Which Copy Do We Get?

Our `binarySearch` stops **as soon as** `a[mid] == target`. With duplicates, that is some copy, not necessarily the first.

index	0	1	2	3	4
`a`	1	2	2	2	3

`binarySearch(a, 2)`: `lo = 0`, `hi = 4`, `mid = 2`, and `a[2] == 2`. It returns **2**, the *middle* copy.

Analogy. You ask a librarian for "a book by this author" and she hands you the one from the middle of the shelf. If you wanted the *first* one, or *how many* there are, you need a different question.

A Better Question: `lowerBound`

Ask for the **first index `i` with `a[i] >= target`**, or `a.size()` if none.

```
int lowerBound(const std::vector<int>& a, int target) {
    int lo = 0, hi = static_cast<int>(a.size()); // range [lo, hi)
    while (lo < hi) {
        int mid = lo + (hi - lo) / 2;
        if (a[mid] < target) { lo = mid + 1; }
        else                 { hi = mid; }          // mid may be the answer
    }
    return lo;
}
```

- The range is **half-open**: `[lo, hi)`, and `hi` is *not* part of it. That is why the loop is `lo < hi`.
- `hi = mid` is correct here: when `a[mid] >= target`, `mid` *might* be the first such index, so we must keep it. In ordinary binary search, a failed comparison meant `mid` was not the target, so `mid` could be dropped.
- It never stops early. It keeps shrinking until `lo == hi`.

Trace: `lowerBound(a, 7)`

`a = {4, 7, 7, 7, 9, 12}`, so `a.size() = 6`.

Step	lo	hi	mid	a[mid]	Decision
1	0	6	3	7	$7 \geq 7$, so `hi = mid = 3`
2	0	3	1	7	$7 \geq 7$, so `hi = mid = 1`
3	0	1	0	4	$4 < 7$, so `lo = mid + 1 = 1`
	1	1			`lo == hi`: return **1**

The result is the index of the **first** 7. Even when `a[mid] == 7`, the search keeps moving left to see whether an earlier 7 exists.

Counting Copies With Two Calls

`a = {4, 7, 7, 7, 9, 12}`

Call	Returns	
`lowerBound(a, 7)`	1	(the first 7)
`lowerBound(a, 8)`	4	(first element ≥ 8 is the 9)
`lowerBound(a, 13)`	6	(`= a.size()`: every element is smaller)

How many 7s? `lowerBound(a, 8) - lowerBound(a, 7) = 4 - 1 = 3`.

Why it works. The first call finds where the 7s *start*; the second finds where the elements *bigger than 7* start. Everything in between is a 7. (This uses integers: "at least 8" means "greater than 7".)

Cost: on a sorted array, two binary searches take $O(\log n)$, no matter how many copies there are. A full linear scan takes $O(n)$.

The Standard Library Already Has It

For a sorted `std::vector<int> b = {4, 7, 7, 7, 9, 12};` (include `<algorithm>`):

```
auto first = std::lower_bound(b.begin(), b.end(), 7); // first >= 7
auto after = std::upper_bound(b.begin(), b.end(), 7); // first > 7
int idx   = first - b.begin();                        // 1
int count = after - first;                            // 3
bool has8 = std::binary_search(b.begin(), b.end(), 8); // false
```

- `lower_bound` and `upper_bound` return **iterators**. For a `std::vector`, subtracting two iterators gives the number of elements between them.
- `binary_search` only answers *yes or no*.
- `std::equal_range` returns both ends at once.
- All of them require the range to be **sorted** by the same ordering.

Practice

`a = {4, 7, 7, 7, 9, 12}`

1. What do `lowerBound(a, 4)`, `lowerBound(a, 3)` and `lowerBound(a, 12)` return?
2. Use two calls to count the 9s in `c = {1, 9, 9, 9, 9, 20}`.
3. Why would changing `hi = mid;` to `hi = mid - 1;` break `lowerBound`? (Try `a = {4, 5}`, target 5.)

Practice

`a = {4, 7, 7, 7, 9, 12}`

1. What do `lowerBound(a, 4)`, `lowerBound(a, 3)` and `lowerBound(a, 12)` return?
2. Use two calls to count the 9s in `c = {1, 9, 9, 9, 9, 20}`.
3. Why would changing `hi = mid;` to `hi = mid - 1;` break `lowerBound`? (Try `a = {4, 5}`, target 5.)

1. **0** (the 4 is the first element ≥ 4), **0** (everything is ≥ 3), **5** (the 12 is at index 5).
2. `lowerBound(c, 10) - lowerBound(c, 9) = 5 - 1 = 4`.
3. `a = {4, 5}`, target 5: `lo = 0`, `hi = 2`, `mid = 1`, and `a[1] >= 5`, so the code sets `hi = mid - 1 = 0`. The range `[0, 0)` is empty and it returns **0**, but the correct answer is **1**. The index `mid` *might* be the answer, so it must stay in the range: `hi = mid`.